Experiment : 9

Text Preprocessing and Feature Scaling Using Python

Mapped CO(s) : CO2

Aim

To perform text preprocessing, feature extraction, normalization, and feature scaling techniques for machine learning applications using Python.

Objectives

- To preprocess textual data by removing unwanted characters and stop words.
- To perform text normalization using tokenization and stemming/lemmatization.
- To convert text into numerical features using vectorization techniques.
- To apply feature scaling methods such as Normalization and Standardization.
- To prepare data for machine learning algorithms.

Prerequisites

- Basic knowledge of Python programming.
- Basic understanding of Natural Language Processing (NLP).
- Python 3.x installed.
- Jupyter Notebook or Google Colab.
- Required libraries: Pandas, NumPy, NLTK, Scikit-learn, and Matplotlib.

Theory/Background

Text preprocessing is an important step in Natural Language Processing (NLP). Raw text often contains punctuation, special characters, stop words, and inconsistent formatting, which can negatively affect machine learning models. Text preprocessing improves the quality of text data before feature extraction.

Common text preprocessing steps include:

- Converting text to lowercase.
- Removing punctuation and special characters.
- Tokenization.
- Removing stop words.
- Stemming or Lemmatization.

After preprocessing, the text is converted into numerical form using feature extraction techniques such as **Bag of Words (CountVectorizer)** or **TF-IDF (Term Frequency–Inverse Document Frequency)**.

Feature scaling is applied to numerical features so that all features have a similar scale. The two commonly used techniques are:

- **Normalization (Min-Max Scaling):** Scales values between 0 and 1.
- **Standardization (Z-score Scaling):** Transforms data to have a mean of 0 and a standard deviation of 1.

These preprocessing techniques improve the performance and accuracy of machine learning algorithms.

Algorithm/Methodology

- Import the required Python libraries.
- Create or load a text dataset.
- Convert text to lowercase.
- Remove punctuation and stop words.
- Perform tokenization and stemming.
- Convert text into numerical features using TF-IDF.
- Normalize and standardize numerical features.
- Display the transformed data.

Dataset Description

Dataset Used: Sample Text Dataset

Description:

A small collection of text sentences is used to demonstrate text preprocessing, feature extraction, and feature scaling techniques.

Attribute	Description
Text	Input text sentence for preprocessing

Program/Implementation

The implementation should include:

- Importing required libraries.
- Loading or creating the text dataset.
- Performing text preprocessing.
- Applying TF-IDF vectorization.
- Performing normalization and standardization.
- Displaying the transformed output.

Test Cases

Test Case	Expected Result
Load text data	Dataset loaded successfully
Remove stop words	Stop words removed
Apply TF-IDF	Feature matrix generated
Normalize features	Values scaled between 0 and 1
Standardize features	Features standardized successfully

Experimental Results

Observation

- Text data was successfully preprocessed.
- Stop words and punctuation were removed.
- TF-IDF converted text into numerical feature vectors.
- Feature scaling techniques normalized and standardized the numerical values.
- The processed data became suitable for machine learning applications.

Result

Text preprocessing, feature extraction, normalization, and feature scaling were successfully performed using Python, preparing the dataset for machine learning tasks.

Conclusion

The experiment demonstrated the importance of text preprocessing and feature scaling in machine learning. The text was cleaned, converted into numerical features using TF-IDF, and scaled using normalization and standardization techniques. These preprocessing steps improve the quality of data and enhance the performance of machine learning models.

Exercises/Assignments

1. Apply CountVectorizer to the same text dataset.
2. Compare TF-IDF and CountVectorizer outputs.
3. Perform lemmatization instead of stemming.
4. Apply Min-Max Scaling to another numerical dataset.
5. Compare Normalization and Standardization results.